



《AI大模型Ragent项目》——四分类撑不住20个知识库为什么要设计意图树

来自： 拿个offer-开源&项目实战

 马丁

2026年04月27日 22:26

开篇引言

上一篇讲完了查询改写，Ragent 用一次 LLM 调用同时完成了改写和拆分，输出一个 RewriteResult——改写后的完整问题加上子问题列表。每个子问题都是独立的、完整的、对检索友好的查询。

子问题列表有了，下一步做什么？意图识别——判断每个子问题应该去哪里找答案。

来看一个场景。假设你在一家电商平台做 AI 客服，平台规模不小，知识库有好几块：商品信息、售后维修、退换货政策是商品服务方向的；国内物流的配送方式和运费规则、跨境物流的海外仓和清关流程是物流方向的。除了知识库，还接了一个订单查询的 MCP 工具，用户问“我的快递到哪了”得去调物流接口。另外还有欢迎问候、介绍助手身份这类系统交互。

用户发来一句：

跨境包裹清关一般要多久？

你作为开发者，一眼就知道该去跨境物流的清关流程知识库搜。但系统怎么知道？

基础系列里学过四种意图类型：知识检索、工具调用、闲聊对话、引导澄清。套用四分类，这个问题会被分到知识检索。没问题，但接下来呢？你有五六（可能几十）个知识库，查哪个？四分类不告诉你。

这就是本篇要解决的核心问题：**当知识库数量多了，扁平的四分类不够用，需要一棵意图树来做精准路由。**

本篇是意图识别子系列的第 1 篇，共 5 篇（第 5~9 篇）。本篇聚焦意图树的设计动机和数据结构——为什么需要树、树长什么样、节点上挂了什么信息、怎么存怎么读。后续第 6 篇讲怎么让大模型给节点打分，第 7 篇讲多子问题多意图的封顶算法，第 8 篇讲歧义引导，第 9 篇讲意图到检索的映射。

扁平分类的天花板

1. 四分类能做什么

基础系列的意图识别篇讲过，用户发来的消息大致可以分成四种意图：

意图类型	含义	典型问题
知识检索	去知识库里搜答案	iPhone 16 Pro 的退货政策是什么？
工具调用	调外部接口拿实时数据	帮我查一下订单 2024112801 的物流进度
闲聊对话	打招呼、闲扯、感谢	你好、谢谢你
引导澄清	问题太模糊，需要反问	我想退货（退哪个产品？）

知识库只有一两个的时候，这四种分类完全够用。判断出是知识检索，直接去那个知识库搜就行；判断出是工具调用，去调那个工具。分类的粒度和知识库的数量刚好匹配。

2. 知识库多了就不够用了

问题出在知识库从 2 个变成 20 个的时候。

继续用电商客服的场景。平台有商品信息、售后维修、退换货政策、国内物流配送方式、国内物流运费规则、跨境物流海外仓、跨境物流清关流程、跨境物流运费计算……光这些就有八九个知识库了。如果再加上不同品类（3C 数码、家电、服装）各自的专属知识库，轻轻松松到 20 个。

四分类告诉你这个问题属于知识检索，然后呢？

- 不知道该查哪个知识库。四分类的知识检索只有一个大类，所有知识库都归这一类。跨境包裹清关一般要多久？和iPhone 16 Pro 的保修期多长？都是知识检索，但该去完全不同的知识库搜。
- KB 和 MCP 混在一起也区分不了。你有订单查询的 MCP 工具，也有物流知识库。用户问我的快递到哪了该调 MCP 工具，用户问你们支持哪些快递公司该查知识库。四分类把 MCP 工具只归为一类工具调用，但你有多 MCP 工具的话，调哪个？
- 同一个框架里处理不了多种类型。知识检索走一套逻辑，工具调用走另一套，闲聊走第三套。四分类做完之后还需要一层路由来决定具体走哪条路径，分类和路由是割裂的。

3. 暴力全库搜索为什么不行

最直觉的方案：不管这么多，所有知识库都搜一遍，最后合在一起排序。

算一笔账。20 个知识库各取 Top-5，合计 100 条候选 chunk。Reranker 要对这 100 条做 Cross-Encoder 精排，计算量大约是单知识库的 20 倍。延迟从几十毫秒飙升到几百毫秒甚至秒级，Token 消耗也跟着翻倍。

更麻烦的是结果质量。不同知识库的内容领域差异很大，跨境物流的 chunk 和 3C 数码的 chunk 放在一起做全局排序，Reranker 容易被迷惑——一段关于国际运费的文档和一段关于手机保修的文档，在语义上可能和用户问题都沾点边，但只有一个是真正相关的。全局排序容易把不相关的 chunk 排到前面，最终答案质量反而下降。

用一句话概括：**扁平分类解决了做什么的问题，但解决不了去哪找的问题。**

为什么是树——三级意图结构

1. 树形结构的设计直觉

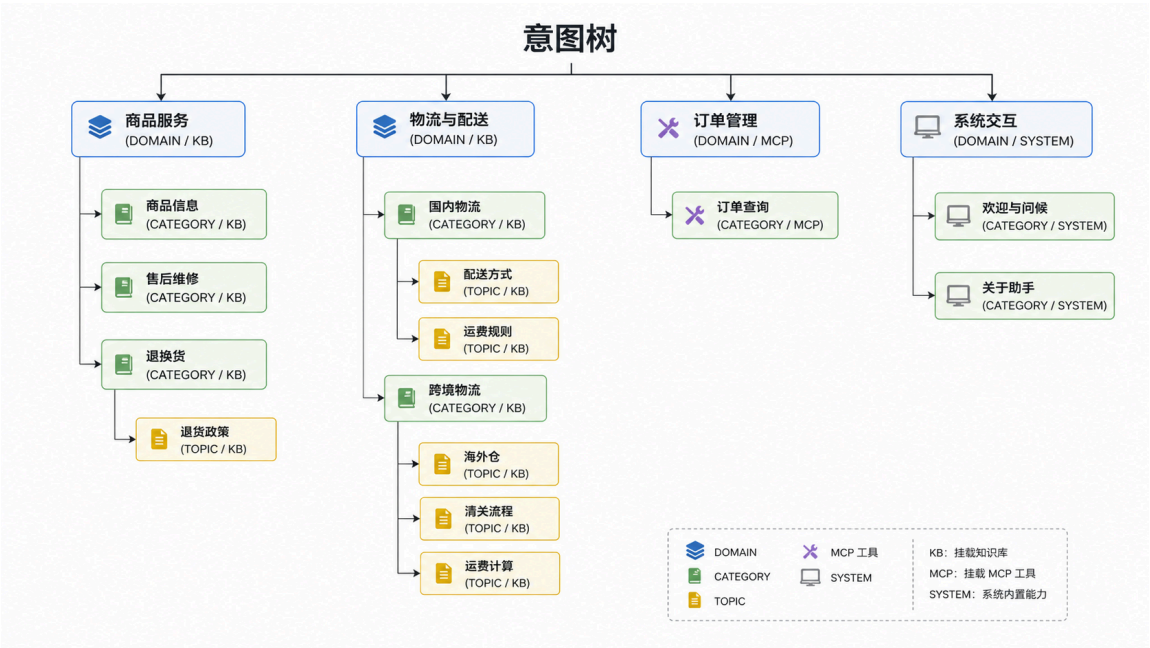
打个比方，电商平台的商品类目是怎么组织的？一级类目（数码、家电、服装）→ 二级类目（手机、耳机、电视）→ 三级类目（iPhone、华为、小米）。用户搜东西的时候，先按大方向分流，再按细方向定位。

意图识别也是一样的思路。用户的问题先按大方向分流——这是商品相关还是物流相关？再按细方向定位——是国内物流还是跨境物流？最后定位到具体知识方向——是配送方式还是运费规则？

层层递进，从粗到细，每一层都把候选范围缩小一半以上。这就是树形结构的核心价值——**把一个大的匹配问题拆成多层小的匹配问题。**

2. 三级结构：DOMAIN → CATEGORY → TOPIC

Ragent 用三级树来组织意图：



三层各司其职：

- DOMAIN（领域层）：** 最粗的分类维度，对应一个大的业务板块。商品服务、物流与配送、订单管理、系统交互——一眼就能看出业务的大方向。DOMAIN 层主要用于在管理后台做顶级分组，不直接参与匹配。如果领域层下没有子节点，会参与匹配
- CATEGORY（分类层）：** 在领域内做进一步细分，通常对应一个具体的服务方向。比如物流与配送下面分了国内物流和跨境物流，商品服务下面分了商品信息、售后维修、退换货。
- TOPIC（主题层）：** 最细粒度，对应一个具体的知识方向。跨境物流下面分了海外仓、清关流程、运费计算三个 TOPIC。

有一点要注意：**不是所有分支都一定有三层。** 商品信息、售后维修在 CATEGORY 层就是叶子节点了——它们的知识库内容比较集中，没必要再往下拆。而跨境物流下面的知识有多个明确方向（海外仓、清关、运费），需要再分一层 TOPIC 才能精准路由。

层数取决于知识库的粒度需求，不是强制三层。两层到叶子的分支和三层到叶子的分支可以同时存在。

3. 只有叶子节点参与匹配

这是整棵树最重要的一条规则：**大模型打分时只看叶子节点，中间层仅用于组织和导航。**

为什么这样设计？

第一，中间层没有对应的执行目标。叶子节点是最终要路由到的目标——一个具体的 Collection、一个 MCP 工具、或一个系统回复模板。而商品服务（DOMAIN）或者跨境物流（CATEGORY）这些中间层节点，命中了也不知道该干什么。

第二，如果中间层也参与打分，分数的含义会变得模糊。假设 LLM 给国内物流（CATEGORY）打了 0.8 分，给配送方式（TOPIC）打了 0.7 分，给运费规则（TOPIC）打了 0.6 分——该走哪个？走国内物流？但国内物流不是一个可执行的目标，下面还有两个 TOPIC。走配送方式？但国内物流的分数更高。逻辑就乱了。

第三，减少 LLM 需要评估的节点数。上面那棵树有 16 个节点，但叶子只有 11 个。只评估叶子，Prompt 更短，推理更快，成本更低。

所有叶子节点会被序列化后发给 LLM，LLM 对每个叶子返回一个 0~1 的分数。具体怎么序列化、Prompt 怎么设计，第 6 篇展开。

4. 三种意图类型共存一棵树

回头看那棵意图树，商品服务和物流的叶子节点是 KB 类型（知识库检索），订单查询是 MCP 类型（工具调用），欢迎与问候和关于助手是 SYSTEM 类型（系统直答）。三种类型放在同一棵树里，用同一套机制做意图识别。

Ragent 定义了三种 IntentKind：

IntentKind	含义	叶子节点的关键字段	命中后的处理方式
KB	知识库检索	collectionName (Milvus Collection)	去对应的 Collection 做向量检索
MCP	工具调用	mcpToolId (MCP 工具 ID)	触发 MCP 工具调用链
SYSTEM	系统直答	promptTemplate (可选)	跳过检索，直接调 LLM 或返回预设回复

```
public enum IntentKind {
    KB(0),      // 知识库类，走 RAG 检索
    SYSTEM(1),  // 系统交互类，如欢迎语、介绍自己
    MCP(2);     // MCP 工具调用，实时数据交互

    private final int code;
}
```

统一框架的好处是：**不需要先判断类型再判断具体意图——一步到位**。LLM 在一次打分中同时考虑所有叶子节点，不管它是 KB、MCP 还是 SYSTEM。用户问你你好，SYSTEM 类型的欢迎与问候节点会拿到高分；用户问我的订单到哪了，MCP 类型的订单查询节点会拿到高分；用户问清关流程，KB 类型的清关流程节点会拿到高分。一棵树搞定所有意图路由。

IntentNode：一个节点上挂了什么

1. 核心字段逐个解读

IntentNode 是意图树的基本单元，每个节点上挂的信息按功能分成四组：

```
@Data
@Builder
public class IntentNode {

    // ===== 标识与层级 =====
    private String id;           // 唯一标识，如 "product-info"、"logistics-overseas-customs"
    private String kbId;        // 知识库 ID
    private String name;        // 展示名称，如「商品信息」「清关流程」
    private String description;  // 语义说明，用于 LLM 匹配
    private IntentLevel level;   // 层级：DOMAIN(0) / CATEGORY(1) / TOPIC(2)
    private String parentId;     // 父节点 ID，根节点为 null

    // ===== 匹配辅助 =====
    private List<String> examples; // 示例问题，帮助 LLM 更精准对齐
    private List<IntentNode> children; // 子节点列表
    private String fullPath;     // 全路径，如「物流与配送 > 跨境物流 > 清关流程」

    // ===== 类型与路由 =====
    private IntentKind kind;      // KB(0) / SYSTEM(1) / MCP(2)
    private String collectionName; // Milvus Collection 名称 (KB 节点专属)
    private String mcpToolId;     // MCP 工具 ID (MCP 节点专属)

    // ===== 检索与生成 =====
    private Integer topK;         // 节点级检索 TopK，未配置时用全局默认
    private String promptSnippet; // 短规则片段（可选）
    private String promptTemplate; // 完整 Prompt 模板（可选）
    private String paramPromptTemplate; // 参数提取提示词 (MCP 专属)

    // ===== 辅助方法 =====
```

```
public boolean isLeaf() {
    return children == null || children.isEmpty();
}

public boolean isKB()      { return kind == null || kind == IntentKind.KB; }
public boolean isMCP()     { return kind == IntentKind.MCP; }
public boolean isSystem() { return kind == IntentKind.SYSTEM; }
}
```

逐组看一下。

1.1 标识与层级

- id: 业务唯一标识，命名规范是 {domain}-{category}-{topic}，比如跨境物流的清关流程节点 ID 是 logistics-overseas-customs。这个 ID 不是数据库主键，是人为定义的语义化标识，方便在日志和调试中一眼看出是哪个节点。
- name: 展示名称，也是 LLM 理解这个节点的第一线索。清关流程、运费规则、商品信息——名称本身就带语义。
- level: 层级枚举，DOMAIN (0)、CATEGORY (1)、TOPIC (2)。
- parentId: 父节点 ID，根节点为 null。通过这个字段把扁平的节点列表组装成树。
- fullPath: 全路径字符串，比如「物流与配送 > 跨境物流 > 清关流程」。在日志打印和歧义引导时特别有用——告诉用户这个节点在树上的完整位置。

```
public enum IntentLevel {
    DOMAIN(0),    // 顶层：商品服务 / 物流与配送
    CATEGORY(1),  // 第二层：商品信息 / 国内物流 / 跨境物流
    TOPIC(2);     // 第三层：配送方式 / 运费规则 / 清关流程

    private final int code;
}
```

1.2 匹配相关

- description: 语义说明，告诉 LLM 这个节点覆盖哪些问题范围。比如清关流程节点的 description 是跨境物流的清关申报、关税计算、禁运品规则等相关说明。这是 LLM 做匹配的核心依据——description 写得好不好直接影响匹配准确率。
- examples: 示例问题列表，帮助 LLM 理解什么样的问题应该匹配到这个节点。比如清关流程节点的 examples 是 ["海淘包裹清关一般要多久? "]。Few-shot 的思路，给 LLM 看几个样本比纯文字描述更直观。
- children: 子节点列表，树形结构的承载字段。

1.3 路由相关

- kind: 意图类型，KB / SYSTEM / MCP。命中后走哪条处理路径完全由这个字段决定。
- collectionName: KB 节点绑定的 Collection 名称。命中这个叶子节点后，搜索引擎直接去这个 Collection 搜。一个叶子节点对应一个 Collection，精准路由。
- mcpToolId: MCP 节点绑定的工具 ID。命中后触发对应的 MCP 工具调用。
- topK: 节点级的检索 TopK。有些知识库内容丰富，可能需要取 Top-10 才够；有些知识库精简，Top-3 就够了。这个字段允许每个叶子节点配自己的 TopK，没配就用全局默认值。

1.4 生成相关

- promptTemplate: 节点专属的 Prompt 模板。比如退货政策节点可以配一个专门的格式化输出要求，让 LLM 按退货流程的步骤来回答。大部分节点不需要配，用通用的 Prompt 模板就行。
- promptSnippet: 短规则片段，会注入到通用 Prompt 中。比如某个节点需要加一条回答时务必提醒用户保修凭证有效期，写在 snippet 里就行，不用写整个模板。一般应用于单次问答匹配到了多个意图，没办法使用节点专属 Prompt 模板，但是有些规则又需要适配的场景。
- paramPromptTemplate: MCP 节点专属，用于从用户问题中提取工具调用的参数。比如订单查询工具需要订单号，这个模板告诉 LLM 怎么从用户问题里抽取订单号。

2. isLeaf() 的判定规则

```
public boolean isLeaf() {
    return children == null || children.isEmpty();
}
```

叶子节点的判定纯粹看 children 是否为空，不看 level。一个 CATEGORY 节点如果没有子节点，它就是叶子。比如商品信息 (CATEGORY) 下面没有 TOPIC 子节点，它直接作为叶子挂知识库，CATEGORY 层就是终点。

这意味着树的深度不是固定的三层——有些分支两层就到叶子，有些分支三层。灵活性换来的是每个分支可以按实际需要决定细分到什么粒度。售后维修的知识集中在一个库里，CATEGORY 就是叶子；跨境物流的知识分散在多个方向，需要再拆一层 TOPIC。

3. 一个具体节点的完整画像

用跨境物流下的清关流程节点为例，看看一个叶子节点的完整字段值：

```
IntentNode.builder()
    .id("logistics-overseas-customs")
    .kbId("1997857139737882625")
    .name("清关流程")
    .level(IntentLevel.TOPIC)
    .parentId("logistics-overseas")
    .kind(IntentKind.KB)
    .description("跨境物流的清关申报、关税计算、禁运品规则等相关说明")
    .examples(List.of("海淘包裹清关一般要多久? "))
    .collectionName("kb_1997857139737882625")
    .build();
```

这些字段在意图识别链路中各有分工：

- description 和 examples 告诉 LLM 什么问题该匹配到这里——LLM 拿到所有叶子节点的 description 和 examples，给每个节点打一个 0~1 的分数。
- collectionName 告诉索引引擎命中后去哪个 Collection 搜——分数最高的叶子节点拿到路由权，系统直接去它绑定的 kb_1997857139737882625 做向量检索。
- fullPath（加载时自动填充为「物流与配送 > 跨境物流 > 清关流程」）用于歧义引导——当多个叶子节点的分数接近时，展示给用户选择。

一个节点同时承载了匹配信息、路由信息、生成信息三层职责。意图树不只是一个分类结构，它是整个 RAG 链路的路由表。

持久化与缓存：意图树怎么存、怎么读

1. 数据库存储：扁平化 → 树形结构

意图树持久化在 PostgreSQL 的 t_intent_node 表里。核心设计是：数据库里存的是扁平化（每行一个节点），通过 parent_code 字段表达父子关系，加载时在内存中重建树形结构。

建表语句的关键字段：

```
CREATE TABLE t_intent_node (
    id                VARCHAR(20) NOT NULL PRIMARY KEY,
    kb_id             VARCHAR(20),
    intent_code       VARCHAR(64) NOT NULL,    -- 业务唯一标识，如 product-info
    name              VARCHAR(64) NOT NULL,
    level             SMALLINT NOT NULL,       -- 0=DOMAIN, 1=CATEGORY, 2=TOPIC
    parent_code       VARCHAR(64),             -- 父节点的 intent_code
    description       VARCHAR(512),
    examples          TEXT,                    -- JSON 数组字符串
    collection_name   VARCHAR(128),           -- Milvus Collection (KB 节点)
    top_k             INTEGER,
    mcp_tool_id       VARCHAR(128),           -- MCP 工具 ID
    kind              SMALLINT NOT NULL DEFAULT 0, -- 0=KB, 1=SYSTEM, 2=MCP
    prompt_snippet    TEXT,
    prompt_template   TEXT,
    param_prompt_template TEXT,
    sort_order        INTEGER NOT NULL DEFAULT 0,
    enabled           SMALLINT NOT NULL DEFAULT 1,
    create_time       TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    update_time       TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
deleted SMALLINT NOT NULL DEFAULT 0);
```

几个值得注意的设计：

- intent_code 是业务语义标识（如 product-info、logistics-overseas-customs），id 是数据库主键（雪花 ID）。在意图树的内存模型中，IntentNode.id 对应的是数据库的 intent_code，不是数据库的 id。这个映射关系在加载时完成。
- parent_code 指向父节点的 intent_code，根节点的 parent_code 为 null。通过这个字段可以从扁平化重建树形结构。
- examples 存储为 JSON 数组字符串，如 ["七天无理由退货怎么申请?", "退货运费谁承担?"]。
- enabled 字段支持动态启停节点。停用后的节点不参与意图匹配——数据库查询时直接过滤 enabled=1。
- deleted 是逻辑删除标志。

为什么不用嵌套 JSON 存整棵树？三个原因。扁平化可以对单个节点做增删改查，不需要每次都读写整棵树。可以用 SQL 做条件查询，比如查所有 KB 类型的叶子节点。MyBatis-Plus 对扁平表的支持也更好——一个 selectList 就能拿到所有节点。

2. Redis 缓存：整棵树的 JSON 快照

每次用户提问都要做意图分类，每次意图分类都要拿到完整的叶子节点列表。如果每次都查数据库，即使有索引，频繁的 DB 查询加上内存组装树形结构的开销也不划算。所以 Ragent 用 Redis 缓存了整棵意图树。

```
@Component
public class IntentTreeCacheManager {

    private static final String INTENT_TREE_CACHE_KEY = "ragent:intent:tree";
    private static final long CACHE_EXPIRE_DAYS = 7;

    // 从 Redis 获取意图树缓存
    public List<IntentNode> getIntentTreeFromCache() {
        String cacheJson = stringRedisTemplate.opsForValue().get(INTENT_TREE_CACHE_KEY);
        if (cacheJson == null) return null;
        return objectMapper.readValue(cacheJson, new TypeReference<>() {});
    }

    // 将意图树保存到 Redis (JSON 序列化整棵树)
    public void saveIntentTreeToCache(List<IntentNode> roots) {
        String cacheJson = objectMapper.writeValueAsString(roots);
        stringRedisTemplate.opsForValue().set(INTENT_TREE_CACHE_KEY, cacheJson, 7, TimeUnit.DAYS);
    }

    // 清除缓存——在意图节点增删改时调用
    public void clearIntentTreeCache() {
        stringRedisTemplate.delete(INTENT_TREE_CACHE_KEY);
    }
}
```

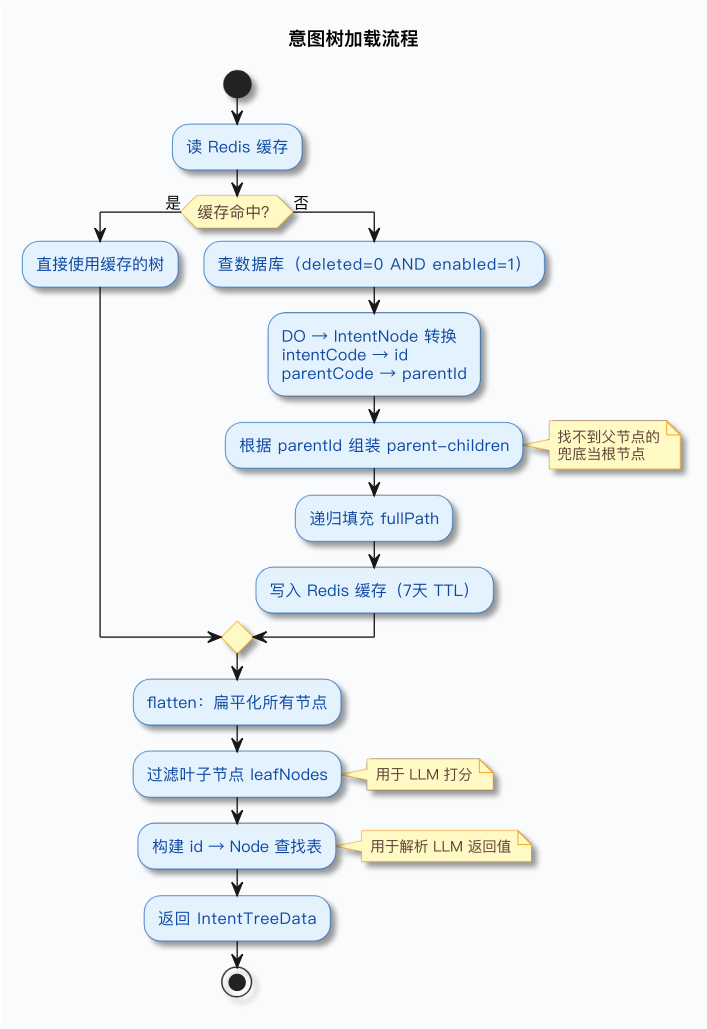
缓存策略很直接：

- 缓存 Key：ragent:intent:tree，一个 Key 存整棵树。
- 过期时间：7 天 TTL。
- 序列化方式：Jackson 把整棵树（含层级关系）序列化为 JSON 字符串。
- 读取策略：先 Redis → 未命中 → 查 DB 重建树 → 写入 Redis。
- 失效策略：管理后台的增删改操作主动调用 clearIntentTreeCache()。

为什么要缓存整棵树而不是缓存单个节点？因为意图分类需要一次性拿到所有叶子节点来构建 Prompt。如果缓存单个节点，就需要用 keys 或者 scan 遍历所有缓存 Key 再逐个取出来组装，不如直接缓存整棵树来得高效。一次 get 拿到一个 JSON 字符串，反序列化就是完整的树，干净利落。

3. 加载流程：从 Redis 到内存结构

意图树的加载由 DefaultIntentClassifier.loadIntentTreeData() 负责，完整流程如下：



核心代码：

```
private IntentTreeData loadIntentTreeData() {
    // 1. 先从 Redis 读取缓存
    List<IntentNode> roots = intentTreeCacheManager.getIntentTreeFromCache();

    // 2. 缓存未命中，从数据库加载并写入缓存
    if (CollUtil.isEmpty(roots)) {
        roots = loadIntentTreeFromDB();
        if (!roots.isEmpty()) {
            intentTreeCacheManager.saveIntentTreeToCache(roots);
        }
    }

    // 3. 构建内存结构
    List<IntentNode> allNodes = flatten(roots); // 扁平化所有节点
    List<IntentNode> leafNodes = allNodes.stream() // 过滤出叶子节点
        .filter(IntentNode::isLeaf).collect(toList());
    Map<String, IntentNode> id2Node = allNodes.stream() // 构建 ID -> 节点的快速查找表
        .collect(toMap(IntentNode::getId, n -> n));

    return new IntentTreeData(allNodes, leafNodes, id2Node);
}
```

加载完成后，内存中会有三个结构：

内存结构	内容	用途
allNodes	所有节点的扁平列表	遍历、调试
leafNodes	仅叶子节点	构建 LLM 打分 Prompt，只需要叶子

id2Node	ID → 节点的 Map	LLM 返回节点 ID 后快速查找对应节点
---------	--------------	-----------------------

4. 从数据库重建树的过程

loadIntentTreeFromDB() 是整个加载链路中最关键的一步——把数据库里的扁平行变成内存中的树形结构：

```
private List<IntentNode> loadIntentTreeFromDB() {
    // 1. 查出所有未删除且已启用的节点（扁平结构）
    List<IntentNodeDO> doList = intentNodeMapper.selectList(
        Wrappers.lambdaQuery(IntentNodeDO.class)
            .eq(IntentNodeDO::getDeleted, 0)
            .eq(IntentNodeDO::getEnabled, 1)
    );

    // 2. DO → IntentNode, 放到 Map 里
    Map<String, IntentNode> id2Node = new HashMap<>();
    for (IntentNodeDO each : doList) {
        IntentNode node = BeanUtil.toBean(each, IntentNode.class);
        node.setId(each.getIntentCode()); // 数据库的 intentCode → 内存的 id
        node.setParentId(each.getParentCode()); // 数据库的 parentCode → 内存的 parentId
        node.setMcpToolId(each.getMcpToolId()); // BeanUtil 未必映射所有字段，显式补设
        node.setParamPromptTemplate(each.getParamPromptTemplate());
        if (node.getChildren() == null) { // 确保 children 不为 null, 避免后面 add NPE
            node.setChildren(new ArrayList<>());
        }
        id2Node.put(node.getId(), node);
    }

    // 3. 根据 parentId 组装 parent → children
    List<IntentNode> roots = new ArrayList<>();
    for (IntentNode node : id2Node.values()) {
        if (node.getParentId() == null || node.getParentId().isBlank()) {
            roots.add(node); // 没有父节点 → 根节点
        } else {
            IntentNode parent = id2Node.get(node.getParentId());
            if (parent == null) {
                roots.add(node); // 找不到父节点 → 兜底也当根节点，避免节点丢失
            } else {
                parent.getChildren().add(node); // 追加到父节点的 children
            }
        }
    }

    // 4. 递归填充 fullPath
    fillFullPath(roots, null);
    return roots;
}
```

整个过程分四步：

1. 从数据库查出所有 deleted=0 且 enabled=1 的节点，是一个扁平的列表。
2. 把每个数据库行转成 IntentNode 对象，关键是 intentCode → id、parentCode → parentId 的映射。注意 BeanUtil.toBean 不一定能映射所有字段（比如 mcpToolId），需要显式补设；children 也要手动初始化为空列表——@Builder.Default 的默认值只在用 Builder 构造时生效，BeanUtil 走的是无参构造 + setter，不会触发默认值。
3. 遍历所有节点，按 parentId 找到父节点，把自己加到父节点的 children 列表里。没有父节点的就是根节点。
4. 递归填充 fullPath，从根到叶逐层拼接，比如「物流与配送 > 跨境物流 > 清关流程」。

第 3 步有一个兜底处理：如果某个节点的 parentCode 在数据库中找不到对应的父节点（可能是父节点被删了或者数据不一致），不会丢弃这个节点，而是兜底当作根节点。这样至少保证所有节点都不会在加载过程中丢失。

5. 缓存与 CRUD 的协同

管理后台提供了完整的意图树 CRUD 接口：

GET	/intent-tree/trees	→ 获取完整意图树
POST	/intent-tree	→ 创建节点
PUT	/intent-tree/{id}	→ 更新节点
DELETE	/intent-tree/{id}	→ 删除节点
POST	/intent-tree/batch/enable	→ 批量启用
POST	/intent-tree/batch/disable	→ 批量停用
POST	/intent-tree/batch/delete	→ 批量删除

所有写操作都遵循同一个模式——先改数据库，再清缓存：

```
// 创建节点
public String createNode(IntentNodeCreateRequest req) {
    // 校验 intentCode 不重复
    // 校验 TOPIC 级别的 KB 节点必须指定知识库
    // collectionName 由 kbId 自动查询填充，创建请求中不用传
    this.save(node);
    intentTreeCacheManager.clearIntentTreeCache(); // 清缓存
}

// 更新节点
public void updateNode(String id, IntentNodeUpdateRequest req) {
    this.updateById(node);
    intentTreeCacheManager.clearIntentTreeCache(); // 清缓存
}

// 删除节点
public void deleteNode(String id) {
    this.removeById(id);
    intentTreeCacheManager.clearIntentTreeCache(); // 清缓存
}
```

下次意图分类时读 Redis 未命中，自动从数据库重新加载最新数据，写入缓存。

创建 KB 节点时，请求中只需要传 kbId（知识库 ID），系统会自动查询该知识库对应的 Collection 名称并填入 collectionName。管理员不需要关心底层向量存储的命名规则——选好知识库就行，collectionName 的映射由系统适配。

这种写时失效、读时重建的策略简单可靠。意图树的变更频率很低（通常是管理员偶尔调整），而读取频率很高（每次用户提问都要读），所以缓存命中率非常高。不需要搞复杂的双写一致性方案。

6. 批量操作的子节点校验

批量停用和批量删除有一个额外的校验逻辑——子节点一致性检查。

举个例子。管理员要停用跨境物流（CATEGORY），但它下面的海外仓、清关流程、运费计算三个 TOPIC 叶子节点还是启用状态。如果只停用了跨境物流但不停用下面的叶子，会出什么问题？

意图分类时 LLM 看的是叶子节点。海外仓、清关流程、运费计算作为叶子节点还在启用状态，LLM 还是能看到它们，还是会给它们打分。用户问了跨境物流的问题，这些叶子节点命中了，系统去对应的知识库检索——但管理员以为跨境物流已经下线了。

所以系统做了一个约束：**停用或删除父节点时，必须把下面的子节点一起操作。**如果批量停用的节点里有父节点，但它下面的某些已启用子节点没有包含在本次操作列表中，操作会被拒绝。防止出现父停子不停的不一致状态。

```
public void batchDisableNodes(List<String> ids) {
    // 如果停用的节点有已启用的子节点未包含在本次操作中，拒绝操作
    // 防止出现父节点停用但子节点还在匹配的不一致状态
    intentTreeCacheManager.clearIntentTreeCache();
}
```

Ragent 还支持从工厂类一键初始化意图树。IntentTreeFactory 用硬编码构建了一棵默认的意图树，新环境部署时调用 initFromFactory() 就能把数据写入数据库：

```
// 从工厂类初始化默认意图树
public int initFromFactory() {
    List<IntentNode> roots = IntentTreeFactory.buildIntentTree();
    List<IntentNode> allNodes = flatten(roots);
    // 逐个创建节点（跳过已存在的 intentCode）
    ...
}
```

这只是初始化用的，正式运行时从数据库加载。管理员后续通过管理后台增删改节点，不会再碰 IntentTreeFactory 的代码。

小结

回顾一下本篇的核心要点：

- 1. 扁平四分类只能解决做什么，解决不了去哪找。当知识库从 2 个增长到 20 个，四分类的知识检索只有一个大类，分不清该查哪个知识库。全库搜索成本太高，结果质量也差。
- 2. 三级意图树（DOMAIN → CATEGORY → TOPIC）把粗粒度到细粒度的匹配逐层拆解。叶子节点直接绑定目标——知识库 Collection、MCP 工具、或系统回复。树的深度不固定，每个分支按需决定细分到哪一层。
- 3. 三种意图类型（KB / MCP / SYSTEM）统一在一棵树里。用同一套 LLM 打分机制做识别和路由，不需要先判断类型再判断具体意图，一步到位。
- 4. IntentNode 是树的基本单元。包含标识信息（id / level / parentId）、匹配信息（description + examples）、路由信息（collectionName / mcpToolId）、生成信息（promptTemplate），一个节点同时承载匹配、路由、生成三层职责。
- 5. 数据库存扁平，Redis 缓存整棵树，写操作主动失效，读时自动重建。简单可靠的缓存策略，意图树变更频率低、读取频率高，缓存命中率非常高。

意图树搭好了，叶子节点有 11 个。但 LLM 怎么知道用户的问题该匹配哪个叶子节点？所有叶子节点的 description 和 examples 要怎么组织成一个 Prompt？LLM 返回的分数格式怎么约定？怎么保证打分结果的确定性和可解析性？下一篇讲意图分类的 Prompt 模板设计——怎么让大模型同时给 30 个意图节点打分。